



20. Genetic Algorithms

Solving optimization problems with algorithms inspired by biology

[Previous Section:](#)

 [19. Gaussian Naive Bayes](#)

Contents:

- [1. The main components of Genetic Algorithms](#)
- [2. Operators on Genetic Algorithms](#)
- [3. The One-Max Problem with Genetic Algorithms](#)
- [4. Crossover in Genetic Algorithms](#)
- [5. Permutation Problem with Genetic Algorithms](#)

[Summary](#)

[References](#)

This section is about Genetic Algorithms, an evolutionary algorithm, and if that sounds a little intimidating, don't worry. Once we walk through it together, you will notice something quite familiar hiding underneath all the fancy biological terms.

The main idea is surprisingly elegant. Instead of solving a problem directly, we *search* for good solutions through mechanisms inspired by natural evolution. Just like how species evolve over generations to better adapt to their environment, Genetic Algorithms evolve solutions step by step, gradually improving them until they become "good enough", or sometimes, surprisingly optimal.

Think about it this way. In traditional algorithms, we often try to construct the perfect answer with logic and rules. But what if the problem is too complex? What if the search space is enormous? Genetic Algorithms take a different path — they don't try to be perfect from the start. Instead, they begin with a *population* of random guesses and let them compete, combine, and mutate over time.

Compete, combine, and mutate. Sounds like biology already, right?

Each candidate solution is treated like an individual in a population. The better it performs, the higher its chance to "survive" and pass its traits to the next generation. Over many generations, weak solutions fade away, strong ones dominate, and entirely new

possibilities emerge through mutation. It is not just computation, it is evolution happening inside your program.

And here comes the beautiful part: You don't always need to know the exact path to the solution. You only need a way to *evaluate* how good a solution is. The algorithm will take care of the rest.

So in this section, we are not just learning another algorithm. We are learning how to let solutions *evolve*.

1. The main components of Genetic Algorithms

Genetic Algorithms are computational search and optimization techniques inspired by natural evolution, but let's not rush past that idea too quickly.

Instead of trying to solve a problem directly, Genetic Algorithms take a step back and ask a different question: ***"What if we let good solutions emerge... instead of constructing them?"***

They don't explore every possible solution one by one—that would be far too slow. Instead, they guide the search toward better answers by mimicking how nature improves species over time → through selection, reproduction, and variation.

So rather than a single solution, we work with a *population* of possibilities. And over time, this population evolves.

Now let's break down what actually makes this work.

- **Population — where everything begins:** Imagine you are given a problem, and instead of guessing one answer, you generate many random ones.
 - This collection is called the *population*. Each individual inside it represents a possible solution, usually encoded as a chromosome (a string of values—binary, integers, or real numbers).
 - At the beginning, these solutions are often not very good. But that's okay → because evolution hasn't started yet.
- **Fitness Function — judging what is "good":** Now comes the key idea. If solutions are going to evolve, we need a way to evaluate them.
 - This is the *fitness function*. It tells us how good each solution is:

Higher fitness → better solution → higher chance to survive

- Think of it like a scoring system. The algorithm doesn't need to know *how* to solve the problem, it only needs to know how to *measure* a solution. And that's enough.
- **Selection — survival of the fittest:** Once every individual has a fitness score... we begin selection.
 - Here, stronger individuals are more likely to be chosen as parents.
 - We are not picking randomly anymore—we are biasing the process toward better solutions.

- This reflects a very simple but powerful idea: Good solutions should have more influence on the future.
- **Crossover — combining good ideas:** Now things get interesting. Instead of creating new solutions from scratch, we take two parent solutions and combine them.
 - This process is called *crossover*. It allows useful traits from different parents to mix together. Sometimes, one parent is good at one part of the problem, while the other excels somewhere else.
 - When combined, the offspring might inherit the best of both. Not always—but often enough to matter.
- **Mutation — introducing randomness:** But if we only combine existing solutions, we might get stuck.
 - So we introduce *mutation*. Small, random changes are applied to offspring.
 - Most of the time, they don't help, but occasionally? They create something entirely new—and sometimes better.
 - Mutation is what keeps the algorithm from becoming too predictable. It maintains diversity, and prevents the population from getting trapped in poor solutions.
- **Generations — evolution over time.** Now we repeat the process:
 Selection → Crossover → Mutation → New population
 - Again, and again, and again. Each cycle is called a *generation*. And with each generation, something subtle happens:
 Weak solutions disappear → Stronger ones dominate → New variations emerge
 - Over time, the population improves.
- **Convergence — when evolution settles.** Eventually, the algorithm reaches a point where improvements become small, or stop entirely.
 - This is called *convergence*. At this stage, we either accept the best solution found, or stop based on a condition (like reaching a maximum number of generations).
 - And here's the important part: We don't always get the perfect solution. But we often get something *very close*—and in many complex problems, that's exactly what we need.

So if you step back and look at the whole picture. Genetic Algorithms are not about solving problems step by step. They are about *evolving solutions over time*. And this makes them especially powerful in situations where the search space is huge, the problem is complex, or traditional methods simply struggle to find an answer. In those cases, instead of forcing a solution, we let it *emerge*.

2. Operators on Genetic Algorithms

Genetic Algorithms rely on a set of core operators that guide the evolutionary process, and this is where things start to feel *less like programming*, and more like *nature at work*.

If the population is the “world” of solutions, then these operators are the forces that shape it over time. Let’s walk through them carefully.

(1) Selection Operator — choosing who survives

The main idea of selection is simple but very powerful: ***Not all solutions deserve to continue.***

Each individual in the population has a fitness score. Some perform well, others don’t. So instead of treating them equally, we give preference to the better ones.

Fitter individuals are more likely to be selected as parents, meaning they get the chance to pass their “genes” to the next generation. This is the algorithm’s version of *survival of the fittest*.

But here’s something subtle. We don’t always pick the absolute best individuals every time. If we did, the population would quickly lose diversity.

So instead, we use strategies like:

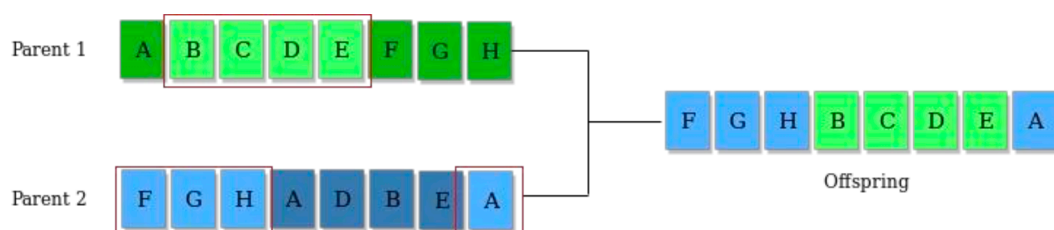
- Roulette Wheel Selection → better individuals have higher probability, not certainty
- Tournament Selection → a small group competes, and the best is chosen
- Rank-Based Selection → individuals are ranked, and selection depends on position

So selection is not just about picking the strongest. It’s about balancing *quality* and *diversity*.

(2) Crossover Operator — combining solutions

Now that we have selected parents, what do we do with them? We let them *reproduce*. The crossover operator simulates mating between individuals. Two parent solutions are chosen, and parts of their chromosomes are exchanged.

Think of it like this: “*Each parent has some good traits... and some not-so-good ones.*”



Crossover tries to combine the good parts together. We randomly choose one or more crossover points along their chromosomes, and swap segments between them.

The result? New individuals called *offspring*. And sometimes these offspring are better than both parents. Not always. But often enough to drive progress.

This is one of the most important ideas in Genetic Algorithms: We don't just improve solutions. We *recombine* them.

(3) Mutation Operator — introducing surprise

But what if all solutions start looking the same? What if the population becomes too "comfortable" with its current state? That's where mutation comes in.

Mutation introduces small, random changes into an offspring's chromosome. A bit flips. A value shifts. Something unexpected happens.



Most mutations don't help. Some even make things worse. But occasionally, a mutation creates something new. Something the algorithm has never seen before. And that "something" might lead to a better solution.

This is how the algorithm avoids getting stuck in local optima—situations where everything looks good, but not the best possible. Mutation keeps the search alive. It pushes the algorithm to explore new regions instead of repeating the same patterns.

If you look at these three operators together:

- Selection → decides *who gets to contribute*
- Crossover → decides *how solutions combine*
- Mutation → decides *how new ideas appear*

And when they work together, generation after generation. Something fascinating happens: The population doesn't just change. It *evolves*.

3. The One-Max Problem with Genetic Algorithms

The One-Max Problem, also known as the MAXONE problem, is one of those rare examples in computer science that looks *almost too simple* at first glance. And yet, it turns out to be one of the best ways to truly understand how Genetic Algorithms work. So let's not underestimate it.

At its core, the One-Max problem is very straightforward: We are given a binary string, a sequence of 0s and 1s. And the goal? Maximize the number of 1s. That's it.

Now you might be thinking: "*Wait... that's the whole problem?*" → Yes. And that's exactly why it's so powerful. Because even though the objective is simple, it allows us to clearly observe how selection, crossover, and mutation work together during evolution.

Think of it like flipping a sequence of coins:

- Each "1" represents a **head** → something desirable.
- Each "0" represents a **tail** → something we want to eliminate.

So the problem becomes: *How do we evolve a sequence that produces as many heads as possible?*

Let's take an example: `0100101`

Not great, but not terrible either. The algorithm doesn't try to fix it directly. Instead, it places this string into a population of many other candidates, and lets evolution take over.

And for a string of length 10, the best possible solution is obvious: `1111111111`

This is the *global optimum*. No ambiguity. No uncertainty. Just the best possible answer. And that's what makes the One-Max problem perfect for studying *convergence*—how a population gradually moves toward an optimal solution.

How the Genetic Algorithm solves it - Now let's walk through how evolution actually happens:

1. **Initialize Population — starting from randomness:** We begin with a population of random binary strings. They are messy. Unstructured. Mostly far from optimal.
 - But that's exactly the point. Because evolution doesn't start from perfection. It starts from randomness.
2. **Fitness Function — measuring quality:** Now we evaluate each string. And here's the beauty of the One-Max problem:
 - The fitness function is incredibly simple, just count the number of 1s. More 1s → higher fitness → better solution.
 - No complex math. No hidden rules. Just a clear signal telling us which solutions are better.
3. **Selection — choosing promising candidates:** Next, we select individuals based on their fitness.
 - Strings with more 1s are more likely to be chosen as parents.
 - But again, not guaranteed. Because we still want some diversity in the population. So we use strategies like tournament selection or roulette wheel selection to balance exploration and exploitation.
4. **Crossover — combining good traits:** Now we take two parent strings, and combine them.
 - For example: `Parent A: 11100010` and `Parent B: 00111101` → After crossover, we might get: `Offspring: 11111101`
 - See what just happened? Good segments from both parents came together... And suddenly, we have a much better solution. This is where real progress often

happens.

5. **Mutation — keeping evolution alive:** But what if the population becomes too similar? What if it gets stuck? That's where mutation comes in.

- Each bit has a small chance of flipping: `0 → 1` or `1 → 0`
- Most of the time, this doesn't help. But occasionally, it introduces a crucial improvement. Or even more importantly—it prevents the algorithm from getting stuck too early.

6. **Repeat — evolution over generations.** Now we repeat the process:

- Selection → Crossover → Mutation → New generation
- Again, and again, and again. And something fascinating starts to happen:
 - The number of 1s increases
 - Weak solutions disappear
 - Strong solutions dominate

→ Until eventually, the population converges toward: `1111111111`

Why this problem matters: The One-Max problem may look simple, but it reveals something deep. It shows how complex behavior can emerge from very simple rules. No direct instructions. No step-by-step solution. Just: **Evaluate** → **Select** → **Combine** → **Mutate**. And over time, improvement happens naturally.

So in a way... the One-Max problem is not just about maximizing 1s. It's about understanding the essence of Genetic Algorithms: We don't force the solution... We let it *evolve*.

Worked Example: One-Max Problem (Manual Simulation): Now, let's slow everything down and actually *watch evolution happen*. Because up until now, Genetic Algorithms might feel a bit abstract. Selection, crossover, mutation, nice ideas. But what do they actually *do*?

So in this example, we're going to simulate the process by hand.

Problem Setup: We want to maximize the number of 1s in a binary string.

- `String length = 5`
- `Population size = 4`
- `Mutation rate = 0.1 (10%)`
- `Generations = 2`

Nothing too large. Just enough to see the mechanics clearly.

Step 1: Initialize Population — starting from randomness

We begin with a random population:

Individual	Binary String	Fitness (# of 1s)
A	10101	3
B	11100	3
C	00111	3
D	00010	1

At this point, everything is still messy. Some solutions are decent. Some are clearly weak. But that's exactly how evolution starts.

Step 2: Evaluate Fitness — measuring quality

Nothing complicated here. We simply count the number of 1s in each string. This is already done in the table above. → And immediately, we notice something:

- A, B, and C are stronger → `fitness = 3`
- D is weaker → `fitness = 1`

→ So naturally, D will struggle to survive.

Step 3: Selection — choosing parents

Now we select individuals for reproduction. Using roulette-wheel selection, fitter individuals have a higher chance of being chosen.

One possible outcome:

- Parent 1 → 10101 (A)
- Parent 2 → 00111 (C)

Notice something important:

We didn't *force* the best pair. We allowed randomness, but biased toward good solutions.

Step 4: Crossover — combining solutions

We perform single-point crossover.

Crossover point = position 2

```
Parent 1: 10 | 101
Parent 2: 00 | 111
```

Now we swap the tails:

Offspring 1: 10 + 111 → 10111
Offspring 2: 00 + 101 → 00101

And just like that, we've created new solutions that didn't exist before.

Step 5: Mutation — introducing variation

Now we apply mutation with probability 10%.

Assume the following:

- 10111 → no change
- 00101 → third bit flips → 01101

So we get:

- Offspring 1 → 10111
- Offspring 2 → 01101

A closer look: Bit-Flip Mutation

Let's pause for a second, because this part is often misunderstood. Mutation doesn't blindly change everything. Each bit has a *small probability* of flipping.

For example, we have the original chromosome:

[1, 0, 1, 1, 0, 0, 1]

Mutation rate = 0.1 → each bit has a 10% chance to flip.

We generate a random number for each bit: **If random < 0.1 → flip; Otherwise → keep**

Example outcome:

Mutated: [1, 1, 1, 1, 1, 0, 1]

Only a few bits change. And that's the key idea: Mutation is *rare, but powerful*.

Step 6: New Generation — survival continues

Now we form the next generation. For simplicity, we replace the old population:

Individual	Binary String	Fitness
New A	10111	4
New B	01101	3
New C	11100	3

Individual	Binary String	Fitness
New D	00010	1

And now... something interesting has happened. We already have a better solution:

→ 10111 (fitness = 4) → Evolution is working.

Generation 2 — let's continue

Selection - We pick the best individuals:

- 10111 (best)
- 11100 (second best)

Crossover (point = 3)

Parent 1: 101 | 11
Parent 2: 111 | 00

Offspring:

Offspring 1: 10100
Offspring 2: 11111

And here it is. A perfect candidate just appeared.

Mutation

- 10100 → mutates → 10101
- 11111 → unchanged

Final Population

Individual	Binary String	Fitness
A	10101	3
B	11111	5
C	10111	4
D	01101	3

And there it is: 11111 → The optimal solution.

What just happened? We didn't construct this solution manually. We didn't "solve" the problem step by step. Instead: **We started with randomness. We selected better solutions. We recombined them. We introduced small variations**

And over just **two generations**, the algorithm *discovered* the best possible answer.

That's the beauty of Genetic Algorithms: They don't search blindly. They don't follow strict rules. They evolve. And sometimes, evolution is faster than logic.

4. Crossover in Genetic Algorithms

Single-Point and Two-Point Crossover *(Two important concepts in Genetic Algorithms)*

Let's zoom in on one of the most powerful ideas in Genetic Algorithms: *Crossover*. Because this is where solutions don't just improve. They *combine*.

At a high level, crossover is about taking two parent solutions, and mixing their "genetic material" to produce offspring. But how exactly do we mix them? That's where different crossover strategies come in.

Let's start with the simplest one.

- **Single-Point Crossover — one cut, one swap**

In single-point crossover, we choose **one position** along the chromosome. And everything after that point gets swapped. That's it.

Let's look at an example:

```
Parent 1: 110 | 10
Parent 2: 001 | 11
```

Crossover point → after position 3

Now we exchange the tails:

```
Child 1: 110 + 11 → 11011
Child 2: 001 + 10 → 00110
```

Simple. Clean. And surprisingly effective.

Here's the intuition:

- The first part comes from one parent
- The second part comes from the other

So each child is a *hybrid*. And sometimes, that hybrid is better than both parents.

- **Two-Point Crossover — swapping the middle**

Now let's make things a bit more interesting. Instead of one cut, we make **two**. This creates a segment in the middle, and that segment gets swapped.

Example:

Parent 1: 1 | 10 | 10
Parent 2: 0 | 01 | 11

Crossover points → positions 2 and 4

Now we exchange the middle parts:

Child 1: (from A) + (from B) + (from A) → 11 01 0 → 11010
Child 2: (from B) + (from A) + (from B) → 00 10 1 → 00101

Now notice what changed. In single-point crossover, we only mix *once*. In two-point crossover, we mix *a segment*.

This allows more flexibility, more diverse combinations, and better chances of combining useful building blocks

So what's the real difference?

Single-point crossover	Two-point crossover
One cut	Two cuts
Simpler	More complex
Faster	Slower
Less mixing	More mixing → More diversity

And here's the deeper idea I want you to see: Crossover is not random chaos. It's a structured way of saying:

“Take what works... from different solutions... and try combining them.”

Because in many problems, the best solution is not created from scratch. It is *assembled*. Piece by piece, from parts that already worked. And crossover? That's the mechanism that makes it happen.

5. Permutation Problem with Genetic Algorithms

Now, up until this point, crossover looked clean and simple. Cut here, swap there, and done. But let me ask you something:

“What if your solution is not just a binary string... but an ordered list where every element must appear exactly once?”

Suddenly, things get tricky.

Why standard crossover fails here - In permutation problems, *order matters*, and *duplicates are not allowed*.

Think about it: If we blindly apply single-point crossover, we might get something like:

- Duplicate cities
- Missing cities
- Completely invalid solutions

And at that point, the algorithm is no longer evolving valid answers. It's just breaking them. This is exactly why we need something smarter. And that's where **Order Crossover (OX)** comes in.

Order Crossover (OX) — preserving structure

Order Crossover is designed specifically for permutation problems. Instead of just swapping bits, it carefully preserves:

- The **relative order** of elements
- The **validity** of the solution

So no duplicates. No missing elements. Everything stays consistent.

Where do we use this? A classic example is the **Traveling Salesman Problem (TSP)**. You are given a set of cities, and distances between them. The goal? Visit each city exactly once, and return to the start, while minimizing total distance.

This is not just about *which* cities. It's about *the order* of visiting them. That makes it a **permutation problem**. And that's exactly where OX shines.

Let's walk through the process - Suppose we have:

Cities: [1, 2, 3, 4, 5, 6, 7, 8]

Parent 1: [1, 2, 3, 4, 5, 6, 7, 8]

Parent 2: [4, 5, 6, 7, 8, 1, 2, 3]

Step 1: Select a crossover segment

We randomly choose two positions. Let's say positions 3 to 5:

- From Parent 1 → [3, 4, 5]
- From Parent 2 → [6, 7, 8]

Step 2: Copy the segment

We copy the segment from Parent 1 into the child:

Child 1: [_ _ 3 4 5 _ _ _]

Step 3: Fill remaining positions (carefully)

Now comes the key idea. We take elements from Parent 2, **in order**, but skip anything already in the child.

```
[4, 5, 6, 7, 8, 1, 2, 3]
```

Already used → [3, 4, 5] ; Remaining → [6, 7, 8, 1, 2]

Now we fill the empty slots from left to right (wrapping around if needed):

```
Child 1: [6, 7, 3, 4, 5, 8, 1, 2]
```

And just like that... A valid permutation.

Step 4: Create the second child

Now we reverse the roles. Start with Parent 2's segment:

```
Child 2: [_ _ 6 7 8 _ _ _]
```

Now fill using Parent 1: Remaining elements → [1, 2, 3, 4, 5]

```
Child 2: [1, 2, 6, 7, 8, 3, 4, 5]
```

Again, valid. No duplicates. No missing cities.

What makes OX special? Look at what just happened. We didn't just mix parents randomly. We preserved:

- A **subsequence** (important structure)
- The **relative order** of remaining elements

This is critical in permutation problems, because structure matters more than individual values.

Advantages

- Simple and intuitive once you see it
- Always produces valid permutations
- Preserves useful ordering information

Disadvantages

- More complex than standard crossover
- Still relies on randomness → not guaranteed optimal
- Requires careful implementation

The deeper idea - Here's what I want you to notice: Not all problems are the same.

- Binary problems → simple crossover works.
- Permutation problems → structure matters → we need smarter operators.

So Genetic Algorithms are not just one method. They *adapt*.

Different problems. Different representations. Different crossover strategies

And Order Crossover? It's a perfect example of evolution, adapting to the problem itself.

Summary

Genetic Algorithms, at first, might feel like just another optimization technique.

But if you've followed everything up to this point, you've probably started to notice something different. They don't solve problems in the traditional sense. They don't follow a fixed sequence of steps to reach an answer → Instead, **they evolve solutions**.

We began with the core idea: A population of candidate solutions, competing, combining, and mutating over time.

And through this process:

- Better solutions are selected
- Useful traits are recombined
- New possibilities emerge through mutation

Until eventually, the population converges toward something strong.

We then explored the key components that make this possible:

- Fitness → how we measure "good"
- Selection → who gets to survive
- Crossover → how solutions combine
- Mutation → how new ideas appear

Each one simple on its own. But together? They create something powerful.

From there, we made it concrete.

- The **One-Max Problem** showed us how a population can gradually move toward an optimal solution, even when starting from randomness.

Step by step, generation by generation. We watched evolution happen.

Then we went deeper into crossover strategies:

- Single-point and two-point crossover → simple, effective mixing
- Order Crossover (OX) → designed for permutation problems

And this revealed an important idea: Genetic Algorithms are not rigid. They adapt to the structure of the problem.

So if you take one thing away from this section, let it be this: **Genetic Algorithms are not about finding the answer directly. They are about creating a system where *better answers emerge over time*.**

But we're not done yet. Because understanding the idea is only the beginning.

Next, we'll bring everything to life:

- We will revisit Genetic Algorithms and implement the **One-Max problem in code**
- And then, take on a more challenging case → the **Knapsack Problem** (both by hand and in code)

And that's where things start to get really interesting. Because once you see evolution working inside your own program. It no longer feels like theory. It feels like something alive.

References

<https://www.geeksforgeeks.org/dsa/genetic-algorithms/>

<https://www.datacamp.com/tutorial/genetic-algorithm-python>

<https://www.youtube.com/watch?v=XP2sFzp2Rig>

Next Section:

 [21. Genetic Algorithms: The Knapsack Problem](#)